

Chuyên đề: Kỹ thuật Euler Tour trên Cây (Euler Tour Technique on Trees - ETT)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

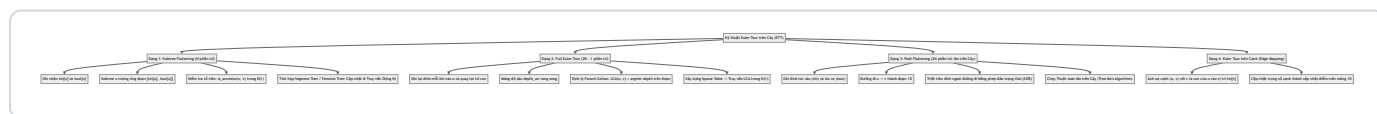
1. Tổng quan & Bản đồ Tư duy (Mindmap)

Trong lý thuyết đồ thị và lập trình thi đấu (Competitive Programming), **Cây (Tree)** là một cấu trúc dữ liệu 2 chiều có tính phân nhánh cao. Các bài toán truy vấn và cập nhật trên cây thường gặp phải rào cản lớn:

- Cây con (subtree) của một đỉnh bao gồm nhiều nhánh con rời rạc.
- Đường đi giữa hai đỉnh (u, v) uốn khúc qua tổ tiên chung gần nhất (LCA).

Kỹ thuật Euler Tour trên Cây (Euler Tour Technique - ETT / Tree Flattening) là một trong những vũ khí tối thượng của cấp độ Two-star. Bằng cách ghi lại nhật ký hành trình duyệt **DFS (Depth-First Search)** kèm theo nhãn thời gian:

- ETT "**trải phẳng**" (**flatten**) toàn bộ không gian 2D của cây thành một **dãy số 1D liên tục**.
- Ánh xạ toàn bộ cấu trúc cây con (subtree) thành một **đoạn liên tục** $[L, R]$, cho phép áp dụng trực tiếp các cấu trúc dữ liệu mạnh mẽ nhất như **Segment Tree**, **Fenwick Tree (BIT)**, và **Thuật toán Mo**.
- Chuyển hóa bài toán tìm **Tổ tiên chung gần nhất (Lowest Common Ancestor - LCA)** thành bài toán **Truy vấn đoạn con nhỏ nhất (Range Minimum Query - RMQ)** với tốc độ truy vấn $O(1)$ **tuyệt đối** bằng Sparse Table.



2. Ba Mô hình Euler Tour Trọng tâm

2.1. Mô hình 1: Subtree Flattening (N phần tử: `tin[u]`, `tout[u]`)

Cơ chế Hoạt động

Chọn một đỉnh bất kỳ làm gốc (thường chọn đỉnh 1). Duy trì một biến đếm thời gian toàn cục `timer`:

- Khi bắt đầu bước chân vào thăm đỉnh u : `tin[u] = ++timer`. Ghi nhận `flat_order[timer] = u`.

- Duyệt đệ quy lần lượt qua tất cả các đỉnh con của u .
- Sau khi đã duyệt xong hoàn toàn 100% các cây con của con cháu u : `tout[u] = timer`.

```
int timer = 0;
vector<int> tin(N + 1), tout(N + 1), flat_order(N + 1);

void dfs_subtree(int u, int p) {
    tin[u] = ++timer;
    flat_order[timer] = u; // Lưu đỉnh tương ứng tại vị trí timer

    for (int v : adj[u]) {
        if (v != p) {
            dfs_subtree(v, u);
        }
    }
    tout[u] = timer;
}
```

Hai Định lý Nền tảng

🚩 Quan trọng:

****Định lý 1 (Ánh xạ Cây con thành Đoạn 1D)**:**

Với mọi đỉnh v nằm trong cây con gốc u (kể cả chính u), thời điểm thăm $tin[v]$ luôn thỏa mãn:

$$tin[u] \leq tin[v] \leq tout[u]$$

Do đó, toàn bộ cây con gốc u tương ứng với ****đoạn liên tục $[tin[u], tout[u]]$ **** trên mảng phẳng `flat_order`.

- Kích thước cây con gốc u : $sz[u] = tout[u] - tin[u] + 1$.

💡 Mẹo / Tối ưu:

****Định lý 2 (Kiểm tra Quan hệ Tổ tiên trong $O(1)$)**:**

Đỉnh u là tổ tiên của đỉnh v khi và chỉ khi khoảng thời gian sống của v nằm trọn vẹn trong khoảng thời gian sống của u :

$$is_ancestor(u, v) \iff tin[u] \leq tin[v] \text{ và } tout[u] \geq tout[v]$$

Thao tác kiểm tra này chỉ mất đúng ****2 phép so sánh số nguyên ($O(1)$)****, không cần đệ quy hay nhảy ngược lên cha!

2.2. Mô hình 2: Full Euler Tour ($2N - 1$ phần tử) & LCA $\rightarrow$ RMQ trong $O(1)$

Bản chất Thuật toán

Thay vì chỉ ghi nhận đỉnh khi bước vào và bước ra, ta ghi lại đỉnh hiện tại **mỗi khi bước vào u và mỗi khi quay trở lại u từ một nhánh con**:

- Mảng `euler` sẽ có kích thước chính xác là:

$$2 \times (N - 1) + 1 = 2N - 1 \text{ phần tử}$$

- Song song với mảng `euler`, ta duy trì mảng `depth_arr` lưu độ sâu của đỉnh tương ứng tại từng bước.

- Ta ghi nhận vị trí xuất hiện đầu tiên của mỗi đỉnh u : `first_occ[u]`.

```
vector<int> euler, depth_arr;
vector<int> first_occ(N + 1);

void dfs_euler_full(int u, int p, int d) {
    first_occ[u] = euler.size();
    euler.push_back(u);
    depth_arr.push_back(d);

    for (int v : adj[u]) {
        if (v != p) {
            dfs_euler_full(v, u, d + 1);
            // Quay trở lại u sau khi thăm xong nhánh v
            euler.push_back(u);
            depth_arr.push_back(d);
        }
    }
}
```

Định lý Farach-Colton & Bender: Chuyển hóa LCA thành RMQ

★ Quan trọng:

****Định lý**:** Với hai đỉnh bất kỳ u và v (giả sử $first_occ[u] \leq first_occ[v]$), đường đi từ u sang v trên dãy Euler bắt buộc phải đi qua đỉnh tổ tiên chung gần nhất $LCA(u, v)$. Hơn nữa, $LCA(u, v)$ chính là đỉnh có ****độ sâu nhỏ nhất**** trên đoạn $[first_occ[u], first_occ[v]]$ của mảng `depth_arr`!

$$LCA(u, v) = euler \left[\underset{first_occ[u] \leq i \leq first_occ[v]}{\operatorname{argmin}} \quad depth_arr[i] \right]$$

Cài đặt Sparse Table giải RMQ trong $O(1)$

Sau khi thu được dãy `depth_arr` độ dài $M = 2N - 1$, ta xây dựng bảng **Sparse Table** trên dãy này:

- Tiền xử lý: $O(M \log M) = O(N \log N)$ thời gian và bộ nhớ.

- Truy vấn: Lấy min đoạn $[first_occ[u], first_occ[v]]$ trong $O(1)$ **tuyệt đối!**

```

int get_lca(int u, int v) {
    int L = first_occ[u], R = first_occ[v];
    if (L > R) swap(L, R);
    int len = R - L + 1;
    int k = 31 - __builtin_clz(len); // log2(len)

    int idx1 = st[k][L];
    int idx2 = st[k][R - (1 << k) + 1];
    return (depth_arr[idx1] < depth_arr[idx2]) ? euler[idx1] : euler[idx2];
}

```

2.3. Mô hình 3: Path Flattening ($2N$ phần tử) & Thuật toán Mo trên Cây

Khi cần giải các bài toán thống kê phức tạp trên đường đi giữa hai đỉnh (u, v) không hỗ trợ gộp nhanh (ví dụ: đếm số màu phân biệt, số lượng phần tử xuất hiện $\geq K$ lần trên đường đi $u \rightarrow v$):

- Ta ghi nhận đỉnh u đúng **2 lần**: Lần 1 khi bước vào ($tin[u] = ++timer$), và lần 2 khi bước ra ($tout[u] = ++timer$). Dãy phẳng có độ dài đúng $2N$.

- **Quy tắc chuyển đường đi $u \rightarrow v$ thành đoạn 1D**:

Giả sử $tin[u] \leq tin[v]$:

1. **Trường hợp 1: u là tổ tiên của v ($LCA(u, v) = u$):**

- Đoạn cần xét trên mảng phẳng là $[tin[u], tin[v]]$.

- Các đỉnh thuộc đường đi $u \rightarrow v$ sẽ xuất hiện **đúng 1 lần** trong đoạn này.

- Các đỉnh thuộc các nhánh con ngoài đường đi sẽ xuất hiện **đúng 2 lần** (vào rồi ra ngay trong đoạn) $\implies$ bị triệt tiêu khi ta dùng cờ đánh dấu trạng thái (Toggle/XOR: đang chọn $\leftrightarrow$ bỏ chọn)!

2. **Trường hợp 2: u không phải tổ tiên của v ($LCA(u, v) = w \neq u$):**

- Đoạn cần xét là $[tout[u], tin[v]]$.

- Các đỉnh trên đường đi xuất hiện đúng 1 lần (các đỉnh ngoài lề xuất hiện 2 lần).

- **Đặc biệt**: Bản thân đỉnh $w = LCA(u, v)$ **không xuất hiện** trong đoạn $[tout[u], tin[v]]$! Ta phải cộng riêng đóng góp của w khi trả lời truy vấn.

2.4. Mô hình 4: Euler Tour trên Cạnh (Edge Mapping)

Trong nhiều bài toán, trọng số nằm trên **cạnh** chứ không phải trên đỉnh.

- **Quy tắc ánh xạ cạnh sang đỉnh**: Trên một cây có gốc, mỗi đỉnh $v \neq \text{root}$ có đúng **một cạnh nối trực tiếp lên cha của nó** ($parent[v], v$).

- Do đó, ta ánh xạ trọng số của cạnh ($parent[v], v$) vào vị trí của đỉnh v , cụ thể là tại vị trí $tin[v]$ trên mảng phẳng ETT!

- **Tính chất**: Toàn bộ các cạnh nằm trong cây con gốc u tương ứng với đoạn $[tin[u] + 1, tout[u]]$ trên mảng ETT (loại trừ chính cạnh đi vào u).

3. Bảng Trace Chi tiết: Cây 5 đỉnh mẫu

Xét cây 5 đỉnh có các cạnh: $(1 - 2), (1 - 3), (2 - 4), (2 - 5)$. Gốc tại 1.



3.1. Dạng 1: Subtree Flattening ($N = 5$)

- Thăm 1: $tin[1] = 1$
- Thăm 2: $tin[2] = 2$
 - Thăm 4: $tin[4] = 3, tout[4] = 3$
 - Thăm 5: $tin[5] = 4, tout[5] = 4$
- Rời 2: $tout[2] = 4$
- Thăm 3: $tin[3] = 5, tout[3] = 5$
- Rời 1: $tout[1] = 5$

Đỉnh u	$tin[u]$	$tout[u]$	Đoạn cây con $[tin, tout]$	Danh sách đỉnh con cháu	Kích thước sz
1	1	5	$[1, 5]$	$\{1, 2, 4, 5, 3\}$ (Toàn bộ cây)	5
2	2	4	$[2, 4]$	$\{2, 4, 5\}$	3
3	5	5	$[5, 5]$	$\{3\}$	1
4	3	3	$[3, 3]$	$\{4\}$	1
5	4	4	$[4, 4]$	$\{5\}$	1

- Kiểm tra tổ tiên:**
- $is_ancestor(1, 4) : tin[1] = 1 \leq tin[4] = 3$ và $tout[1] = 5 \geq tout[4] = 3 \implies$ **Đúng (TRUE)**.
- $is_ancestor(3, 5) : tin[3] = 5 > tin[5] = 4 \implies$ **Sai (FALSE)**.

3.2. Dạng 2: Full Euler Tour ($2N - 1 = 9$ phần tử) & LCA

Dãy duyệt ghi nhận đỉnh và độ sâu:

Chỉ số i	0	1	2	3	4	5	6	7	8
$euler[i]$	1	2	4	2	5	2	1	3	1
$depth[i]$	0	1	2	1	2	1	0	1	0

Mảng xuất hiện đầu tiên: $first_occ[1] = 0, first_occ[2] = 1, first_occ[3] = 7, first_occ[4] = 2, first_occ[5] = 4$.

- Tìm LCA(4, 5):**
- $first_occ[4] = 2, first_occ[5] = 4 \implies$ xét đoạn $[2, 4]$ trong $depth : \{2, 1, 2\}$.

- Giá trị nhỏ nhất là 1 tại chỉ số 3 $\implies euler[3] = 2$.
- Kết luận: $LCA(4, 5) = 2$!
- **Tìm** $LCA(4, 3)$:
- $first_occ[4] = 2, first_occ[3] = 7 \implies$ xét đoạn $[2, 7]$ trong `depth`: $\{2, 1, 2, 1, 0, 1\}$.
- Giá trị nhỏ nhất là 0 tại chỉ số 6 $\implies euler[6] = 1$.
- Kết luận: $LCA(4, 3) = 1$!

4. Ba Case Studies Thực chiến Kinh điển

Case Study 1: Cập nhật & Truy vấn Cây con ([CSES 1137 - Subtree Queries](#))

- **Bài toán:** Cho cây N đỉnh. Có 2 thao tác: (1) Gán giá trị đỉnh $u = x$; (2) Tính tổng các giá trị trong cây con gốc u .
- **Giải pháp:**
- Chạy ETT Dạng 1 tính $tin[u], tout[u]$.
- Thao tác 1: `update(tin[u], delta)` trên Fenwick Tree ($O(\log N)$).
- Thao tác 2: `query(tout[u]) - query(tin[u] - 1)` trên Fenwick Tree ($O(\log N)$).

Case Study 2: Truy vấn Đường đi từ Gốc ([CSES 1138 - Path Queries](#))

- **Bài toán:** Cho cây N đỉnh. Thao tác 1: Gán giá trị đỉnh $u = x$; Thao tác 2: Tính tổng đường đi từ gốc 1 đến u .
- **Giải pháp:**
- Một đỉnh x đóng góp vào đường đi từ gốc đến u khi và chỉ khi x là tổ tiên của u .
- Khi thay đổi giá trị đỉnh x một lượng Δ , toàn bộ các đỉnh thuộc cây con gốc x sẽ tăng tổng đường đi thêm Δ .
- Biến đổi mảng hiệu với ETT Dạng 1: Cộng $+\Delta$ tại vị trí $tin[x]$ và trừ $-\Delta$ tại vị trí $tout[x] + 1$ trên Fenwick Tree.
- Truy vấn tổng đường đi tới u : Chỉ việc lấy tổng tiền tố `query(tin[u])` trong $O(\log N)$!

Case Study 3: Đếm số Màu phân biệt trên Đường đi ([SPOJ COT2 - Count on a Tree II](#))

- **Bài toán:** Cho cây N đỉnh có màu, truy vấn số màu phân biệt trên đường đi giữa u và v .
- **Giải pháp:**
- Dùng ETT Dạng 3 (Path Flattening, $2N$ phần tử).
- Biến mỗi truy vấn (u, v) thành đoạn $[L, R]$ trên mảng 1D:
 - Nếu u là tổ tiên của v : $[tin[u], tin[v]]$.
 - Nếu không: $[tout[u], tin[v]]$ và bổ sung thêm $lca = LCA(u, v)$.
- Áp dụng Thuật toán Mo chia mảng thành các block kích thước $\sqrt{2N}$.
- Duy trì mảng `vis[u]` (trạng thái đỉnh): Khi mở rộng/thu hẹp cửa sổ, nếu đỉnh xuất hiện lần 1 thì thêm màu vào tập; xuất hiện lần 2 thì bỏ màu ra khỏi tập. Độ phức tạp $O((N + Q)\sqrt{N})$.

⚠ 5. Cạm bẫy Phòng thi & Kỹ thuật Tối ưu

1. **Nhầm lẫn kích thước mảng Full Euler Tour ($2N - 1$) và Path Flattening ($2N$):**
 2. Dạng Full Euler Tour có $2N - 1$ phần tử; mảng Sparse Table phải khai báo tối thiểu $2 \cdot MAXN$.
 3. Dạng Path Flattening có $2N$ phần tử; mảng phẳng phải có kích thước $2 \cdot MAXN$. Khai báo $MAXN$ thông thường sẽ dẫn đến lỗi **SIGSEGV / Out of Bounds** ngay lập tức!
 4. **Tràn Call Stack khi Cây suy biến:**
 5. Với cây dạng đường thẳng $N = 2 \cdot 10^5$, đệ quy DFS sẽ có độ sâu $2 \cdot 10^5$. Trên Windows hoặc môi trường giới hạn stack nhỏ (8MB/16MB), chương trình sẽ bị Runtime Error.
 6. Thêm cờ mở rộng stack `-Wl,--stack,268435456` khi biên dịch trên máy hoặc chuyển sang khứ đệ quy bằng `std::vector` mô phỏng Call Stack.
 7. **Quên đỉnh LCA trong Mo trên Cây:**
 8. Trong Dạng 3 khi u không phải tổ tiên của v , đoạn $[tout[u], tin[v]]$ **không chứa đỉnh LCA(u, v)!**
Nếu quên thêm thủ công đóng góp của đỉnh LCA khi chốt đáp án cho truy vấn, kết quả sẽ bị thiếu màu/giá trị dẫn đến **WA**.
 9. **Nhầm lẫn giữa thứ tự DFS ($tin[u]$) và chỉ số đỉnh ban đầu:**
 10. Trên cây phân đoạn / Fenwick Tree, các vị trí thao tác phải là nhãn thời gian $tin[u]$, không bao giờ được dùng trực tiếp u !
-

📖 6. Tài liệu Tham khảo & Đọc thêm

1. **USACO Guide (Gold Section):**
 2. [Euler Tour Technique](#) — Trải phẳng cây thành đoạn tuyến tính $[tin, tout]$.
 3. **VNOI Wiki:**
 4. [Euler Tour trên cây](#) — Ánh xạ cây con và đường đi lên Segment Tree / Fenwick.
 5. **CP-Algorithms:**
 6. [Lowest Common Ancestor - Farach-Colton and Bender](#) — Euler Tour chuyển bài toán LCA thành RMQ trong $O(1)$.
-



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.